

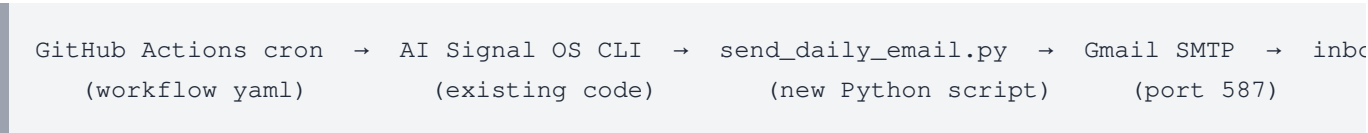
Daily AI News Email — Setup Walkthrough

This doc explains how the daily email pipeline was built on top of the existing AI Signal OS engine. Read it once and you'll be able to debug, extend, or rip it out without re-reading the conversation.

Goal

Send a daily email at **5:00 PM US Central** containing the AI Signal OS weekly brief for a given project — generated freshly from the markdown reports in `reports/` and scored against the project profile.

Architecture (3 layers)



Layer	File	Role
Trigger + orchestration	.github/workflows/daily-email.yml	Runs on schedule, on push to main, or via manual button. Sets up Python, installs deps, runs the engine, calls the email script.
News engine (pre-existing)	<code>__main__.py</code> → <code>aisignal/*.py</code>	Parses markdown reports, scores articles, exports <code>data/exports/project_{id}_brief.md</code>
Email sender (new)	scripts/send_daily_email.py	Reads the brief, converts markdown → HTML, sends multipart email via Gmail SMTP.

Three GitHub repo secrets glue the layers together: `SMTP_USER`, `SMTP_APP_PASSWORD`, optional `EMAIL_TO`, optional `PROJECT_ID`.

How the engine produces the brief (existing code, not changed by setup)

- `python -m AI_news init-db` — creates SQLite at `data/ai_signal_os.db` and seeds three sample projects (id 1, 2, 3).
- `python -m AI_news ingest-dir reports` — `parser.py` regex-extracts articles from every `*.md` in `reports/` (title, URL, source, category). Each is stored as an `ArticleRecord` row.
- `python -m AI_news export-project 1` — `services.build_weekly_brief` calls

`get_recommendations`, which runs `score_article` per article against project #1's constraints (edge/on-device, latency, memory, etc.). It emits a markdown brief at `data/exports/project_1_brief.md` with Top Developments, Best-Fit Tools, Architecture Direction, What to Ignore, and Open Questions.

A small fix was made to the renderer during setup so headlines are clickable markdown links — see "Code changes during setup" below.

How the workflow file works

[.github/workflows/daily-email.yml](https://github.com/workflows/daily-email.yml)

Triggers (three of them):

Event	When it fires
<code>schedule: "0 22 * * *"</code>	22:00 UTC daily = 5:00 PM CDT (March–November) or 4:00 PM CST (November–March). Cron is UTC-only on GitHub.
<code>push to main</code>	Only when paths matching the workflow, scripts, engine code, reports, <code>__main__.py</code> , or <code>requirements.txt</code> change. Keeps random README edits from spamming you.
<code>workflow_dispatch</code>	The manual "Run workflow" button in the Actions tab.

Job steps (in order):

1. **Checkout into `AI_news/` subdir.** Critical detail: the engine is invoked as `python -m AI_news`, which means Python needs to find a *package* literally named `AI_news` on the import path.
`actions/checkout@v4` with `path: AI_news` drops the repo contents into `$GITHUB_WORKSPACE/AI_news/`, satisfying that requirement.
2. **Set up Python 3.11.**
3. **Install deps** from `AI_news/requirements.txt` (`requests`, `beautifulsoup4`, `feedparser`, `pytest`).
4. **Init DB + ingest reports.** Runs from `$GITHUB_WORKSPACE`, so the engine's `Path("data/...")` writes to `$GITHUB_WORKSPACE/data/ai_signal_os.db`. Ephemeral — the runner discards everything when the job ends.
5. **Build weekly brief** for `${{ secrets.PROJECT_ID || '1' }}`. Default project id is 1 (Orin Nano Edge Assistant) — change by setting a `PROJECT_ID` repo secret to 2 (Document AI) or 3 (SaaS Copilot).
6. **Send brief via Gmail SMTP.** The script reads `data/exports/project_<id>_brief.md` and sends it.

Runtime: ~60–90 seconds per run, well within the 10-minute timeout cap.

How the email script works

[scripts/send_daily_email.py](#)

1. **Reads env vars** (`SMTP_USER`, `SMTP_APP_PASSWORD`, optional `EMAIL_TO`, optional `BRIEF_PATH`). Exits with a non-zero code if required secrets are missing, which surfaces as a red X on the Actions run.
2. **Loads the brief markdown** from `data/exports/project_<id>_brief.md`.
3. **Converts markdown** → **HTML** with a tiny in-script function — handles headings, bullets, bold, and `[text](url)` links. No external markdown dep needed.
4. **Builds a multipart email** with both a plaintext body (raw markdown) and an HTML body (styled, 720px max width). Gmail shows the HTML by default; mail clients that don't render HTML fall back to plaintext automatically.
5. **Connects to `smtp.gmail.com:587`**, upgrades to TLS via `starttls()`, logs in with the App Password, and calls `send_message`.

Why Gmail App Password, not OAuth: App Passwords are designed for non-interactive scripts and require only 2FA on the account. OAuth would require hosting a token-refresh flow, which is overkill for a personal cron job.

Setup steps you performed (the parts I can't do)

1. **Made the repo public** so I could read it via WebFetch and clone it locally.
2. **Generated a Gmail App Password** at <https://myaccount.google.com/apppasswords> — Google only shows it once.
3. **Added two GitHub repo secrets** at Settings → Secrets and variables → Actions:
 - `SMTP_USER = darshilshah3008@gmail.com`
 - `SMTP_APP_PASSWORD = 16-char password from step 2`
4. **Merged the first PR** that added the workflow and script.
5. **Clicked Run workflow** once manually to verify end-to-end (the email arrived; that confirmed secrets + SMTP + engine + script all wired up correctly).

Steps 2 and 3 are the parts I can never automate — Google's security model and GitHub's secret encryption both require interactive user action.

Code changes during setup (chronological)

Commit	What	Why
59e063d Add daily email workflow	Added <code>daily-email.yml</code> , <code>send_daily_email.py</code> , <code>gitignore</code> <code>data/</code> .	Initial setup.

5feb4bf Add clickable URLs to the weekly brief	Edited <code>aisignal/services.py</code> so <code>build_weekly_brief</code> renders titles as <code>[**Title**] (url)</code> instead of bare text.	Email had no links — useless.
25ef256 Auto-trigger email workflow on push to main	Added <code>push:</code> trigger with path filter to the workflow.	So fixes auto-send a test email without clicking "Run workflow."
4f0217d Move daily email to 5:00 PM Central	Changed cron from <code>30 1 * * * (7 AM IST)</code> to <code>0 22 * * * (5 PM CDT)</code> .	Your preferred time zone.

Daily reality check

The pipeline runs every day, but **the email content only changes when `reports/` changes**. Right now `reports/` contains 6 markdown files from April 2026 — so every 5 PM email will be the same brief until you add new reports.

To make it actually "daily AI news":

- **Option A — manual:** drop a new `ai_news_YYYY-MM-DD.md` into `reports/` and push. Workflow auto-fires, email arrives with new content.
- **Option B — RSS auto-fetcher:** add a workflow step before "ingest reports" that pulls overnight headlines from feeds (TechCrunch AI, OpenAI Blog, KDnuggets, HF Papers, etc.) using the existing `feedparser` dependency, and writes them into a fresh `reports/ai_news_$(date +%F).md`. Fully automated; not yet built.
- **Option C — Claude-generated digest:** call the Anthropic API at the start of each run to summarize a curated set of feeds. Highest quality output, ~\$0.01/day on Haiku 4.5.

Debugging recipe (when something breaks)

Email didn't arrive:

1. Open https://github.com/darshilshah3008/AI_news/actions/workflows/daily-email.yml — look at the latest run.
2. Red X? Click into the failed step, expand the log.
3. `SMTPAuthenticationError`: App Password is wrong or expired. Regenerate at <https://myaccount.google.com/apppasswords> and update the `SMTP_APP_PASSWORD` secret. App passwords break if you change your Google account password.
4. `brief not found`: the engine step failed earlier. Check the "Build weekly brief" step log — usually a parsing error in a new report file.
5. Email step succeeded but no email: check Gmail Spam / Promotions tabs (first-time SMTP sends sometimes land there until you mark Not Spam).

Test locally without GitHub:

```
cd <parent of AI_news/>      # e.g. E:/Github
python -m pip install -r AI_news/requirements.txt
python -m AI_news init-db
python -m AI_news ingest-dir AI_news/reports
python -m AI_news export-project 1
cat data/exports/project_1_brief.md

# Optional: actually send a test email
SMTP_USER='you@gmail.com' \
SMTP_APP_PASSWORD='xxxx xxxx xxxx xxxx' \
python AI_news/scripts/send_daily_email.py
```

Files added/changed by this setup

.github/workflows/daily-email.yml	(new)	– cron + push + manual triggers, 5 steps
scripts/send_daily_email.py	(new)	– markdown→HTML + Gmail SMTP sender
aisignal/services.py	(edit)	– clickable URLs in 3 brief sections
.gitignore	(edit)	– ignore data/ai_signal_os.db and data/exports
EMAIL_SETUP.md	(new)	– this file

Nothing in the original engine logic (scoring, parsing, ingestion) was touched — only the brief renderer was modified to include URLs that the parser already captured but the original code threw away.